AI Engineer Assignment: Grounded Knowledge Search

Nov 6, 2026 · @Vaibhav Rane

What this assignment is for

This assignment tests how you think about retrieval, not how much you can build. We want to see the decisions you make, the evidence behind each one, and what you do when the evidence surprises you.

Use any AI tools you like; we assume you will. An assistant can write a chunker or a vector query in minutes. It cannot tell you which approach works on this corpus, because it has not seen your results. That judgement is what we are evaluating.

A small system that is measured, honest about its failures and clearly reasoned will beat a large one that ticks every box.

The scenario

Our client is an equity research firm. Its analysts lose hours searching company filings for figures and disclosures. They want an internal assistant that:

- answers questions from the filings and cites the document and page;
- says it does not know when the filings do not contain the answer;
- searches only the companies on the analyst's own desk.

Role	Companies it can see	Example question
consumer	PepsiCo, Best Buy, MGM Resorts, Ulta Beauty	"How did Ulta Beauty's store count change over the last three years?"
health_industrials	Johnson & Johnson, Pfizer, 3M, Boeing	"What were Pfizer's main revenue drivers in 2021?"
tech_telecom	AMD, Adobe, Corning, Verizon	"Did Verizon's total debt rise or fall in 2022?"
lead	All 12 companies	"Compare R&D spending at AMD and Pfizer in 2022."

The corpus

We provide 119 filings from these 12 US-listed companies, covering 2015 to 2024, all as PDFs. They include annual reports (10-K), quarterly reports (10-Q), current reports (8-K) and earnings releases. Many run past 100 pages and are full of financial tables.

- **`pdfs/`**: the 119 filings, named like `3M_2018_10K.pdf`.
- **`manifest.csv`**: each filing's doc_id, company, sector, desk, document type and period.
- **`dev_questions.jsonl`**: 20 sample questions, each with its role, answer, doc_id and the pages that hold the evidence (numbered from 1).

Most companies have a filing for nearly every year, so the same line item appears in many documents with different values. Read the data before you build.

The filings come from the public FinanceBench dataset, whose questions and answers are published online. Tuning your system to them will not help you. Most of our hidden questions are our own, and we compare your results on both sets.

Download: [corpus link to be added before sending]

What to build

Build a system that retrieves from the filings and answers questions with page citations. Then make retrieval measurably better than your own baseline.

1. **Ingestion.** Parse every PDF, including its tables. Chunk the text, embed the chunks and store them in a vector store of your choice. Keep the page number with every chunk. Re-ingesting a changed filing must not create duplicates.
2. **Baseline.** Start with the simplest retrieval that works, and measure it before you improve anything.
3. **Deliberate improvement.** Form a hypothesis about why the baseline fails, change one thing, and measure again. The techniques are your choice. We care whether each one earned its place.
4. **Access control.** A user may only receive chunks, citations and answers from filings of companies on their own desk. The lead role sees every company. Enforce this in retrieval, not only in the prompt. Explain where you enforce it and what that costs.
5. **Grounded answers.** Every claim cites the document and page it came from. When a figure is calculated, such as a margin or a growth rate, show the inputs. When the filings cannot answer a question, refuse rather than guess.
6. **Evaluation.** Build your own eval set of at least 30 questions; you may include our 20. Measure both retrieval and answer quality. You choose the metrics and tell us why they are the right ones.

Constraints

- It must run for free, on local models or free tiers. If you use a paid model, switching to a free one must take one environment variable.
- Treat 5 seconds per answer on an ordinary laptop as a budget. If you go over it, tell us why the extra time was worth it.

Interface contract. We will run our hidden questions through these commands, so match them exactly. Any language is fine. Roles are `consumer`, `health_industrials`, `tech_telecom` and `lead`.

```
ingest <corpus_dir>
search --role <role> --k <n> "<question>"
ask    --role <role> "<question>"
```

`search` prints a JSON list, best match first. `page` is the PDF page number, counted from 1:

```
[{"doc_id": "3M_2018_10K", "page": 60, "chunk_id": "...", "score": 0.0,
"text": "..."}]
```

`ask` prints one JSON object:

```
{"status": "answered | refused", "answer": "...", "citations": [{"doc_id":
"3M_2018_10K", "page": 60, "chunk_id": "..."}]}
```

What not to build

Time spent on any of these earns no credit:

- a web UI (the CLI is the interface);
- an MCP server, an agent framework or tool-calling;
- deployment, or Docker unless it is the easiest way for us to run your code;
- an authentication system (the role is a command-line flag);
- an observability platform (plain logs are fine);
- fine-tuning an embedding model or an LLM.

How to work

We are grading your reasoning, so leave a trail of it as you work.

- **Look at the data before you build.** Record what you noticed about the corpus and how it shaped your design.
- **Keep `DECISIONS.md` as you go, not at the end.** Write one entry per real decision: the options you considered, the evidence, what you chose, and what would change your mind. Most good submissions have 8 to 12 entries.
- **Keep `EXPERIMENTS.md`.** Give each retrieval experiment one row: what changed, your hypothesis, the metrics, and whether you kept it. Include the experiments that did not help. An honest negative result counts in your favour.
- **Commit as you go and do not squash.** Your git history is part of the submission.
- **Write a short `AI_NOTES.md`.** Say how you used AI tools, and describe at least one time an AI tool was wrong or steered you badly, and how you noticed.
- **Handle ambiguity yourself.** When something is unclear, make a reasonable assumption, write it down and keep going. You can email us questions, but we will often reply "your call; write down what you assumed".

Deliverables

The length limits are deliberate. Deciding what to leave out is part of what we assess.

Item	What it contains	Limit
Git repository	Runs on a clean machine with no paid account; a `.env.example`; no committed keys; full commit history	—
`README.md`	How to run it, the architecture (a diagram or one paragraph), your final numbers, known limitations	2 pages
`DECISIONS.md`	Your decision log, written as you went	—
`EXPERIMENTS.md`	One row per experiment, then your 3 worst failures: the question, what was retrieved, the root cause, and what you would try next	—
`eval/`	Your eval set and the script that produces your numbers; we will run it	—
`AI_NOTES.md`	How you used AI tools, and where they went wrong	1 paragraph
Video	At most 1 minute of demo. Use the rest for the decision you are least sure of and the experiment that most changed your mind	5 minutes

Timeline and what happens next

- **Effort:** we scoped this for about 3 focused days. If you pass 4 days, stop and write up what you have, including what you would do next.
- **Deadline:** 5 calendar days after you receive this brief.
- **How to submit:** reply to the email that sent you this brief. Include the repository link (public, or private with access granted to us) and the video link.

After you submit, we run our hidden questions through your system, run your eval script and read your logs.

If you move forward, we hold a 60-minute debrief. You walk us through your system, we discuss your decisions, we change a constraint and talk it through, and you make one small change to your code live. AI tools are allowed in the live part too.

Your work stays yours. Halden is fictional, and we will not deploy or reuse your code.